PROGRAMMING WITH PYTHON

Lekha A

Computer Applications

PES UNIVERSITY
CELEBRATING 50 YEARS

PROGRAMMING WITH PYTHON

Exception Handling, File I/O, Modules and Packages

Object Serialization using Pickle

Lekha A
Computer Applications

PROGRAMMING WITH PYTHON

Serialization

PES UNIVERSITY
CELEBRATING 50 YEARS

- Serialization is the process of converting the object into a format that can be stored or transmitted.
- After transmitting or storing the serialized data, the object can be reconstructed later and the exact same structure/object can be obtained which makes it really convenient to continue using the stored object later on instead of reconstructing the object from scratch.

PROGRAMMING WITH PYTHON

Serialization

PES UNIVERSITY
CELEBRATING 50 YEARS

- Object serialization refers to process of converting state of an object into byte stream.
- Once created, this byte stream can further be stored in a file or transmitted via sockets etc.
- Reconstructing the object from the byte stream is called deserialization.

Serialization

- Python’s terminology for serialization and deserialization is pickling and unpickling respectively.

PROGRAMMING WITH PYTHON
What can be pickled?

PES UNIVERSITY
CELEBRATING 50 YEARS

- None, True, and False
- integers, floating point numbers, complex numbers
- strings, bytes, bytearrays
- tuples, lists, sets, and dictionaries containing only picklable objects
- functions defined at the top level of a module (using def, not lambda)
- built-in functions defined at the top level of a module
- classes that are defined at the top level of a module

PROGRAMMING WITH PYTHON

Use cases for Pickle

PES UNIVERSITY
CELEBRATING 50 YEARS

- Persistence
  - Pickle can be used to save Python objects to a file and load them back later.
  - This is helpful for storing program state, saving data structures, or caching objects.
- Interprocess Communication
  - Pickle facilitates passing Python objects between different processes or threads by converting them into a portable byte stream.

PROGRAMMING WITH PYTHON

Use cases for Pickle

PES UNIVERSITY
CELEBRATING 50 YEARS

- Data Storage
  - It's used in databases or other storage systems where Python objects need to be stored or retrieved without losing their structure.
- Machine Learning and Model Persistence
  - In machine learning, Pickle is often used to save trained models or complex data structures representing models, allowing them to be loaded and used later for predictions or analysis.

PROGRAMMING WITH PYTHON
Considerations when using Pickle

PES UNIVERSITY
CELEBRATING 50 YEARS

- Security Risks
  - Unpickling data from untrusted sources can lead to security vulnerabilities.
  - Maliciously crafted pickle data can execute arbitrary code when unpickled, so it's important to only unpickle data from trusted sources.

PROGRAMMING WITH PYTHON

Considerations when using Pickle

PES UNIVERSITY
CELEBRATING 50 YEARS

- Compatibility
  - Pickle is Python-specific.
  - While it's efficient for storing and loading Python objects, the serialized data may not be compatible with other programming languages.

PROGRAMMING WITH PYTHON
Considerations when using Pickle

PES UNIVERSITY
CELEBRATING 50 YEARS

- Version Dependency
  - Pickle data can be affected by changes in Python versions or changes to the structure of the objects being pickled.
  - This can lead to compatibility issues when unpickling objects in a different environment or Python version.

PROGRAMMING WITH PYTHON
Examples

PES UNIVERSITY
CELEBRATING 50 YEARS

- # Picking and Unpickling

```
import pickle
```

Picking and Unpickling a List

```
my_list = ['apple', 'banana', 'cherry']
with open('list.pkl', 'wb') as file:
    pickle.dump(my_list, file)
with open('list.pkl', 'rb') as file:
    loaded_list = pickle.load(file)
print("Loaded List:", loaded_list)
```

```
Loaded List: ['apple', 'banana', 'cherry']
```

PROGRAMMING WITH PYTHON

Examples

PES UNIVERSITY
CELEBRATING 50 YEARS

Pickling and Unpickling a Dictionary

```
my_dict = {'name': 'Abhinav', 'age': 30, 'city': 'Mangalore'}
with open('dict.pkl', 'wb') as file:
    pickle.dump(my_dict, file)
with open('dict.pkl', 'rb') as file:
    loaded_dict = pickle.load(file)
print("Loaded Dictionary:", loaded_dict)
```

```
Loaded Dictionary: {'name': 'Abhinav', 'age': 30, 'city': 'Mangalore'}
```

PROGRAMMING WITH PYTHON

Examples

PES UNIVERSITY
CELEBRATING 50 YEARS

- ### Picling and Unpickling Custom Object

```
class Person:
    def __init__(self, name, age):
        self.name = name
        self.age = age

    def __repr__(self):
        return f"Person(name={self.name}, age={self.age})"
person = Person('Aryaman', 25)
```

```
with open('person.pkl', 'wb') as file:
    pickle.dump(person, file)
with open('person.pkl', 'rb') as file:
    loaded_person = pickle.load(file)
print("Loaded Person:", loaded_person)
```

```
Loaded Person: Person(name=Aryaman, age=25)
```

PROGRAMMING WITH PYTHON
Examples

PES UNIVERSITY
CELEBRATING 50 YEARS

- ## Pickling and Unpickling Multiple Objects

```
list_data = [1, 2, 3]
dict_data = {'a': 1, 'b': 2}
string_data = "Hello, World!"
with open('multiple.pkl', 'wb') as file:
    pickle.dump(list_data, file)
    pickle.dump(dict_data, file)
    pickle.dump(string_data, file)
with open('multiple.pkl', 'rb') as file:
    loaded_list = pickle.load(file)
    loaded_dict = pickle.load(file)
    loaded_string = pickle.load(file)
print("Loaded List:", loaded_list)
print("Loaded Dictionary:", loaded_dict)
print("Loaded String:", loaded_string)
```

```
Loaded List: [1, 2, 3]
Loaded Dictionary: {'a': 1, 'b': 2}
Loaded String: Hello, World!
```

PROGRAMMING WITH PYTHON
Examples

PES UNIVERSITY
CELEBRATING 50 YEARS

- ### Pickling and Unpickling a function

```
def greet(name):
    return f"Hello, {name}!"
with open('greet_function.pkl', 'wb') as f:
    pickle.dump(greet, f)
with open('greet_function.pkl', 'rb') as f:
    loaded_greet = pickle.load(f)
print(loaded_greet("Amanda"))
```

```
Hello, Amanda!
```

Pickling and Unpickling Pandas Data Frame

```
import pandas as pd
data = {'Name': ['Akshara', 'Bharat', 'Cauvery'],
        'Age': [25, 30, 35],
        'City': ['New York', 'Los Angeles', 'Chicago']}
df = pd.DataFrame(data)
df.to_pickle('dataframe.pkl')
loaded_df = pd.read_pickle('dataframe.pkl')
print(loaded_df)
```

```
      Name  Age         City
0  Akshara   25     New York
1   Bharat   30  Los Angeles
2  Cauvery   35      Chicago
```

PROGRAMMING WITH PYTHON

Advantages of using Pickle

PES UNIVERSITY
CELEBRATING 50 YEARS

- Recursive objects (objects containing references to themselves)
  - Pickle keeps track of the objects it has already serialized, so later references to the same object won't be serialized again.

PROGRAMMING WITH PYTHON

Advantages of using Pickle

PES UNIVERSITY
CELEBRATING 50 YEARS

- Object sharing (references to the same object in different places)
  - This is similar to self-referencing objects.
  - Pickle stores the object once, and ensures that all other references point to the master copy.
  - Shared objects remain shared, which can be very important for mutable objects.

PROGRAMMING WITH PYTHON

Advantages of using Pickle

PES UNIVERSITY
CELEBRATING 50 YEARS

- User-defined classes and their instances
  - Pickle can save and restore class instances transparently.
  - The class definition must be importable and live in the same module as when the object was stored.

PROGRAMMING WITH PYTHON

Recap

PES UNIVERSITY
CELEBRATING 50 YEARS

- Serialization
- What can be pickled?
- Usecases
- Common considerations while using Pickle
- Examples
- Advantages

THANK YOU

Lekha A

Computer Applications